



UNIVERSITÀ
DEGLI STUDI DI BARI
ALDO MORO



Ingegneria del Software

Analisi dei Requisiti

ITPS A.A. 2025/2026

Vita Santa Barletta
Danilo Caivano
Antonio Piccinno

Dipartimento di Informatica - Università degli Studi di Bari
Via Orabona, 4 - 70125 - Bari
Tel: +39.080.5443270 | Fax: +39.080.5442536
serlab.di.uniba.it



REQUISITI

Definizione di Requisiti

- Per l'Institute of Electrical and Electronic Engineering (**IEEE**), i requisiti
 - ❑ Esprimono **capacità** e **condizioni** (**vincoli**) necessarie per **risolvere problemi** o **realizzare obiettivi** di business degli utenti
 - ❑ Sono **documentati** da contratto, standard, specifiche o altri documenti formalmente convenuti che descrivono le Capacità e le Condizioni di sviluppo ed esercizio del software
 - ❑ La loro documentazione, in genere, si basa su una **rappresentazione grafica** delle Capacità e delle Condizioni che descrivono



Definizione di Requisiti

- Il processo di ricerca, analisi e documentazione, e verifica di servizi che il sistema deve fornire e dei suoi vincoli operativi è chiamato **ingegneria dei requisiti**
- Alcuni dei problemi che sorgono durante il processo di ingegneria dei requisiti derivano dalla mancanza di una separazione netta tra i diversi livelli
 - ❑ **Requisiti dell'utente:** indicano i requisiti astratti di alto livello
 - ❑ **Requisiti del sistema:** forniscono la descrizione dettagliata di quello che il sistema dovrebbe fare
- Diversi livelli di specifiche del sistema sono utili, perché comunicano le informazioni che riguardano il sistema a diversi tipi di utilizzatori dei documenti o "lettori"



Esempio: Requisiti sistema Mentcare

- L'esempio, tratto da un sistema informativo medico per la cura di pazienti con problemi di salute mentale (Mentcare), mostra come **un requisito dell'utente** può essere espanso in **diversi requisiti del sistema**

Definizione dei requisiti dell'utente

1. Il sistema Mentcare genererà mensilmente dei rapporti che riportano il costo dei farmaci prescritti da ciascuna clinica durante il mese

Specificazione dei requisiti del sistema

1. Nell'ultimo giorno lavorativo di ogni mese, dovrà essere generata una sintesi dei farmaci prescritti, il loro costo e le cliniche che li hanno prescritti
2. Il sistema genererà il rapporto per la stampa dopo le 17:30 dell'ultimo giorno lavorativo di ciascun mese
3. Sarà creato un rapporto per ciascuna clinica; dovranno essere elencati i nomi dei singoli farmaci, il numero totale di prescrizioni, il numero delle dosi prescritte e il costo totale dei farmaci prescritti
4. Se i farmaci sono disponibili in dosi unitarie differenti (per esempio, 10 mg, 20 mg ecc.), dovranno essere creati dei rapporti separati per ciascuna dose unitaria
5. L'accesso ai rapporti dei costi dei farmaci dovrà essere limitato agli utenti autorizzati, come indicato nella lista di controllo degli accessi



Definizione Operativa dei Requisiti

- I Requisiti sono l'espressione delle proprietà che deve esibire un **Sistema** sviluppato per **risolvere un particolare problema**
- Il **sistema automatizza** totalmente od in parte alcune **attività di business** che deve eseguire un **utente**

Collocazione dei Requisiti

- I Requisiti di Sistema possono essere tutti collocati nel solo software; in tal caso **Requisiti di Sistema = Requisiti Software**
- I Requisiti possono essere soddisfatti da un **sistema Software-Intensivo**:
 - ❑ Una parte consistente dei **Requisiti di Sistema** sono allocati nel software

Ruolo dei Requisiti

- I requisiti servono per far comunicare correttamente
 - ❑ Gli **Stakeholder / Parti Interessate** tra di loro
 - ❑ Portatori di conoscenza del **Dominio Applicativo** con gli Sviluppatori, portatori di conoscenza del **Dominio delle Tecnologie Informatiche**
- **I requisiti servono**, tra l'altro, a:
 - ❑ Definire la portata del progetto
 - ❑ Stimare i costi di produzione
 - ❑ Prevedere le risorse da mettere a disposizione
 - ❑ Schedulare il progetto
 - ❑ Alimentare il progetto del sistema obiettivo
 - ❑ Individuare i casi di test per validare l'intero sistema prodotto e per il collaudo dello stesso
 - ❑ Definire la documentazione e l'addestramento per l'uso corretto del sistema



Parti Interessate: Persone

- Il **Manager/Committente/Customer** determina i vincoli di costo e di personale da utilizzare, requisiti del processo di sviluppo
- L'**Utente** è interessato ai servizi resi dal prodotto, alla loro qualità ed alla loro adattabilità
- Lo **Sviluppatore** è interessato alla componibilità, alla qualità ed alla estensione del patrimonio di base



Parti Interessate: Sistemi

- **Organi di Legislazione o di Controllo** potrebbero essere interessati a comportamenti od a livelli di qualità (es.: controllo della composizione degli scarichi, certificazione della qualità, ...)
- **Legacy System** che deve far parte della linea di prodotto
 - ❑ Potrebbe essere fonte di requisiti per la componibilità
- **Sistemi Cooperanti** con cui i costruendi prodotti devono comunicare
 - ❑ Possono essere fonte di requisiti di interfacce



I Requisiti in Pratica

- I requisiti **non possono esporre tutti i dettagli** perché questi, in genere, sono molti e complessi
- L'analista deve **astrarre la realtà** e descrivere i requisiti **concettualmente**
 - ❑ I dettagli trascurati saranno **gradi di libertà** per le fasi successive
- L'**analisi** deve descrivere la realtà con un **modello concettuale** che rilevi, almeno rigorosamente, tutti e solo **i dettagli che sono vincolanti** nello sviluppo dell'Applicazione analizzata
- L'analisi **non deve** includere dettagli che riguardano la **progettazione o la codifica** (separazione degli interessi)
 - ❑ Questi dettagli potrebbero generare difetti (fault) nel software e/o sprechi nel processo



Esempio di Requisiti come User Stories

esempio Caso di Studio

Ingegneria del Software

Anno Accademico 2020-2021

SERVIZIO PRENOTAZIONE TAMPONI COVID-19

USER STORIES

User Story

- COME <ruolo>
- VOGLIO <fare qualcosa>
- COSÌ CHE <possa ottenere valore per il business>

Come magazziniere *--- ruolo*
voglio poter filtrare l'archivio ordini secondo *--- obiettivo*
la data di ricezione
per consultare gli ordini evasi *--- motivazione*



Esempio User Stories

- Attore: UTENTE

❑ **UT.01**
Come **Voglio** prenotare un tampone
Così che possa avere la possibilità di effettuare il test diagnostico in un laboratorio d'analisi convenzionato
Utente tampone

❑ **UT.02**
Come **Voglio** conoscere il risultato del
Così che possa dimostrare la mia positività al virus
Utente tampone

❑ **UT.03**
Come **Voglio** conoscere in quanto tempo, il costo e la tipologia di tamponi dei laboratori d'analisi
Così che possa prenotare il tampone presso il laboratorio d'analisi
Utente

❑ **UT.04**
Come **Voglio** pagare il costo del
Così che mi venga rilasciata la ricevuta della prenotazione
Utente tampone

Esempio User Stories

- Attore: LABORATORIO DI ANALISI

- ❑ **LA.01**

Come Laboratorio d'analisi
Voglio convenzionarmi
Così che possa far parte del gruppo dei laboratori d'analisi aderenti a questo servizio

- ❑ **LA.02**

Come Laboratorio d'analisi
Voglio visualizzare l'elenco delle prenotazioni presso il mio laboratorio d'analisi
Così che possa sapere quando e a chi dovrò effettuare il tampone

- ❑ **LA.03**

Come Laboratorio d'analisi
Voglio accedere ai questionari di Anamnesi degli utenti
Così che possa conoscere la situazione clinica degli utenti prenotati presso il mio laboratorio d'analisi

- ❑ **LA.04**

Come Laboratorio d'analisi
Voglio fornire un calendario
Così che possa comunicare le disponibilità del laboratorio

REQUISITI FUNZIONALI e NON FUNZIONALI

Requisiti funzionali e non funzionali

I **requisiti dei sistemi software** sono divisi in:

- **Requisiti Funzionali**: sono definizioni di servizi che il sistema deve fornire
 - ❑ Indicano come il sistema dovrebbe reagire a particolari input e come dovrebbe comportarsi in particolari situazioni
- **Requisiti Non Funzionali**: sono vincoli sulle funzioni o servizi offerti dal sistema
 - ❑ Di solito si applicano al sistema completo, non a singole funzioni o servizi



Requisiti funzionali

- I **requisiti funzionali** descrivono ciò che il sistema deve fare
 - a. Un utente deve essere in grado di cercare gli appuntamenti nelle liste di tutte le cliniche
 - b. Il sistema deve generare ogni giorno, per ciascuna clinica, l'elenco dei pazienti che hanno un appuntamento
 - c. Ogni membro del personale che usa il sistema dovrà essere identificato in modo univoco mediante un codice
- Tali requisiti definiscono specifiche funzionalità che devono essere incluse nel sistema

Requisiti non funzionali

- I **requisiti non funzionali** sono requisiti che non riguardano direttamente specifici servizi forniti dal sistema ai suoi utenti
 - ❑ Di solito specificano o limitano le caratteristiche del sistema nel suo complesso
 - ❑ Derivano dalle necessità degli utenti, dai limiti del budget, dalle politiche organizzative, dalla necessità di interoperabilità con altri sistemi software o hardware, da fattori esterni come norme di sicurezza o legislazioni sulla privacy
- Requisiti non funzionali possono derivare da
 - ❑ Caratteristiche richieste dal software (**requisiti del prodotto**)
 - ❑ Dall'organizzazione che sta sviluppando il software (**requisiti organizzativi**)
 - ❑ Da sorgenti esterni (**requisiti esterni**)



Requisiti non funzionali

- **Requisiti del prodotto**

- ❑ Il sistema Mentcare dovrà essere disponibile a tutte le cliniche durante l'orario normale di lavoro (lun-ven 08:30 - 17:30). Il periodo di inattività all'interno dell'orario normale di lavoro non dovrà superare 5 secondi durante la giornata

- **Requisiti organizzativi**

- ❑ Gli utenti del sistema Mentcare dovranno identificarsi utilizzando la carta di identità rilasciata dall'autorità del servizio sanitario

- **Requisiti esterni**

- ❑ Il sistema dovrà implementare le disposizioni sulla privacy dei pazienti definite nello standard HStan-03-2006-priv



Requisiti non funzionali

- *Il personale medico dovrà essere in grado di usare tutte le funzioni del sistema dopo due ore di addestramento; dopodichè, il numero medio di errori commessi dagli utenti esperti non dovrà essere maggiore di due, per ogni ora di utilizzo del sistema*
- Quando è possibile, bisognerebbe descrivere i requisiti non funzionali quantitativamente, in modo che possano essere verificati in maniera oggettiva. Esempio:
 - Velocità
 - transizioni elaborate al secondo
 - tempi di risposta utenti/eventi
 - tempo di refresh dello schermo
 - Robustezza
 - Tempo di riavvio dopo un malfunzionamento
 - percentuale di eventi che causano malfunzionamenti
 - Probabilità di corruzione dei dati dopo un malfunzionamento
 - etc.



PROCESSO di PRODUZIONE DEI REQUISITI

Problemi dei requisiti

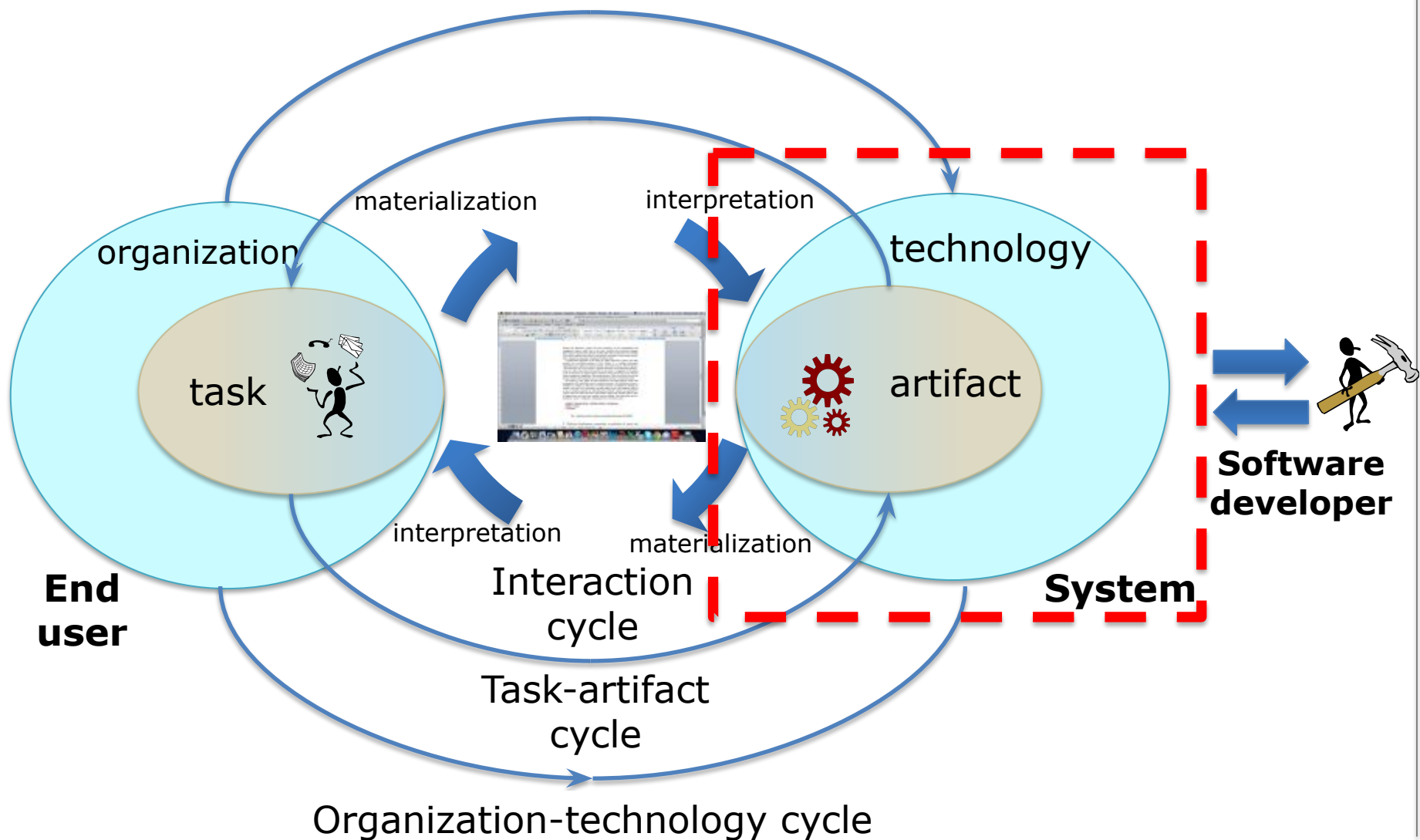
- Gli utilizzatori **non sanno quello che vogliono**
- Gli utilizzatori esprimono ciò che vogliono nel **proprio linguaggio**, secondo **la propria cultura** e con le **loro competenze**
- Differenti utilizzatori dello stesso software possono avere **richieste che confliggono tra loro**
- Durante lo sviluppo del software possono emergere **nuovi utilizzatori** e può **cambiare il dominio applicativo**

Deduzione dei requisiti

- **Scoperta e comprensione dei requisiti:** il processo di interazione con gli stakeholder del sistema per scoprire i loro requisiti
- **Classificazione e organizzazione dei requisiti:** l'attività che, partendo dalla raccolta non strutturata dei requisiti, raggruppa quelli correlati e li organizza in gruppi coerenti
- **Negoziare e priorità dei requisiti:** inevitabilmente, quando sono coinvolti più stakeholder, i requisiti possono essere in conflitto. Questa attività si occupa di dare una priorità ai requisiti, trovare e risolvere i conflitti attraverso la negoziazione
- **Documentazione dei requisiti:** i requisiti vengono documentati. In questa fase possono essere prodotte le prime bozze dei documenti di requisiti software



The Interaction and Co-Evolution (ICE) model



[Costabile, Fogli, Marcante, Piccinno,
Supporting Interaction and Co-evolution of Users and Systems, Proc. AVI
2006]



Transizione tra waterfall e processi agili

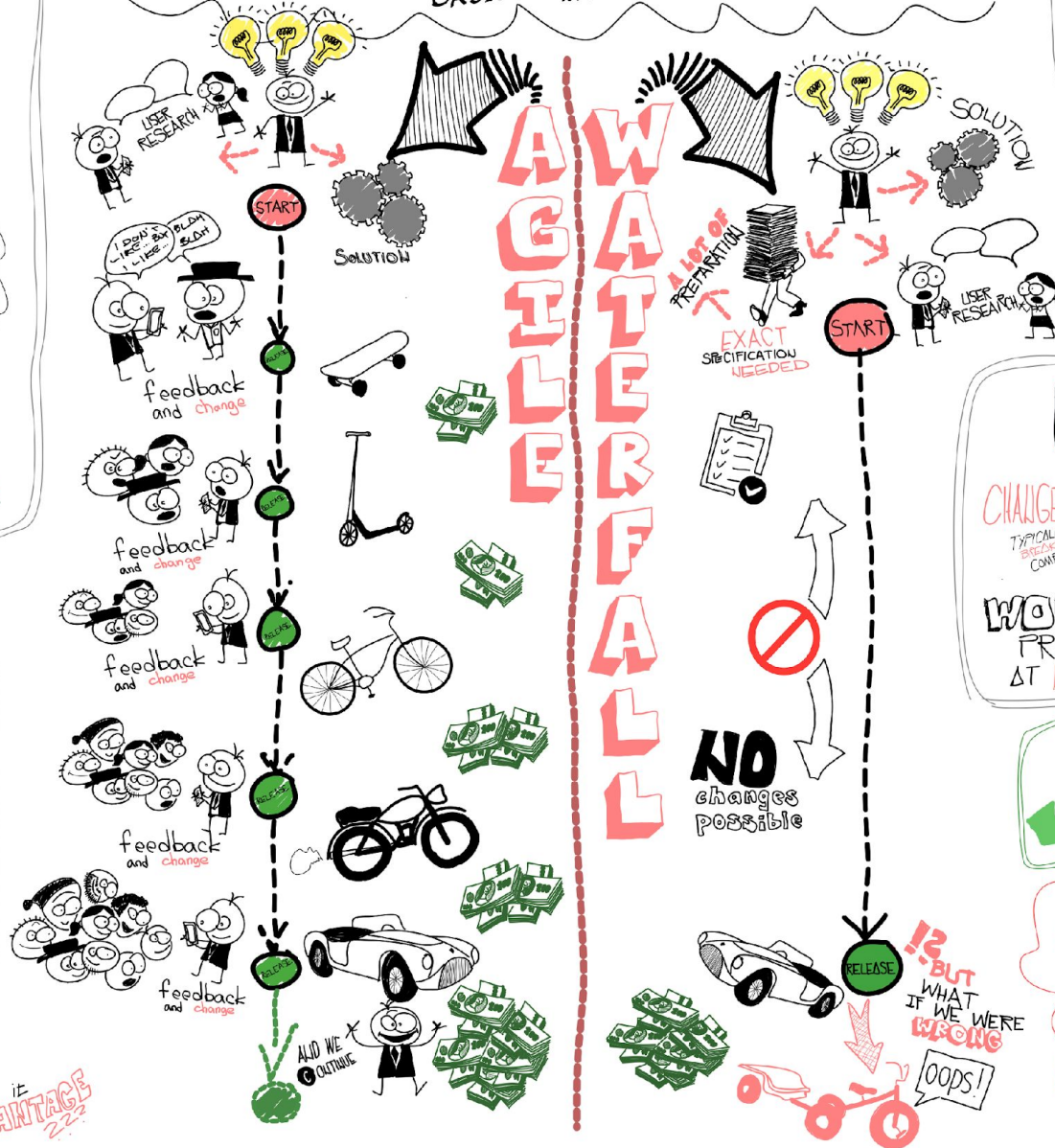
- Nel **waterfall classico** i requisiti sono presentati come il **prodotto della prima fase** e dovevano essere completamente ben definiti prima di passare alle altre fasi dello sviluppo
 - **Terminato lo sviluppo** si può consegnare il software ed avere un parere dai committenti e dagli utilizzatori.
- Si passa ai processi: **iterativi, incrementali, spirale**. In questi i prototipi anticipano la possibilità di avere il parere dai committenti e dagli utilizzatori prima che lo sviluppo sia nel pieno della sua esecuzione
- Si arriva ai **processi agili** dove una **continua consegna** delle parti di software consente di avere più frequenti pareri dai committenti e dagli utilizzatori e di fornire più rapide e soddisfacenti soluzioni alle necessità del committente e degli utenti



AGILE vs WATERFALL

BASIC COMPARISON

www.AGILEWHEEL.COM



EARLY and CONTINUOUS DELIVERY

WELCOME Change Requests



LISTEN TO FEEDBACK

SIMPLICITY

GOAL CAN BE UNKNOWN



Planning is **LESS** concrete

documentation CAN BE **DEPRIVED**

FINAL Product CAN be **DIFFERENT**

BUT Isn't it **ADVANTAGE**



USE MANAGE

Every Stage has **START** and **END**

Progress can be **EASILY** shared

WELL DOCUMENTED SPECIFICATION



CHANGES NOT EXPECTED

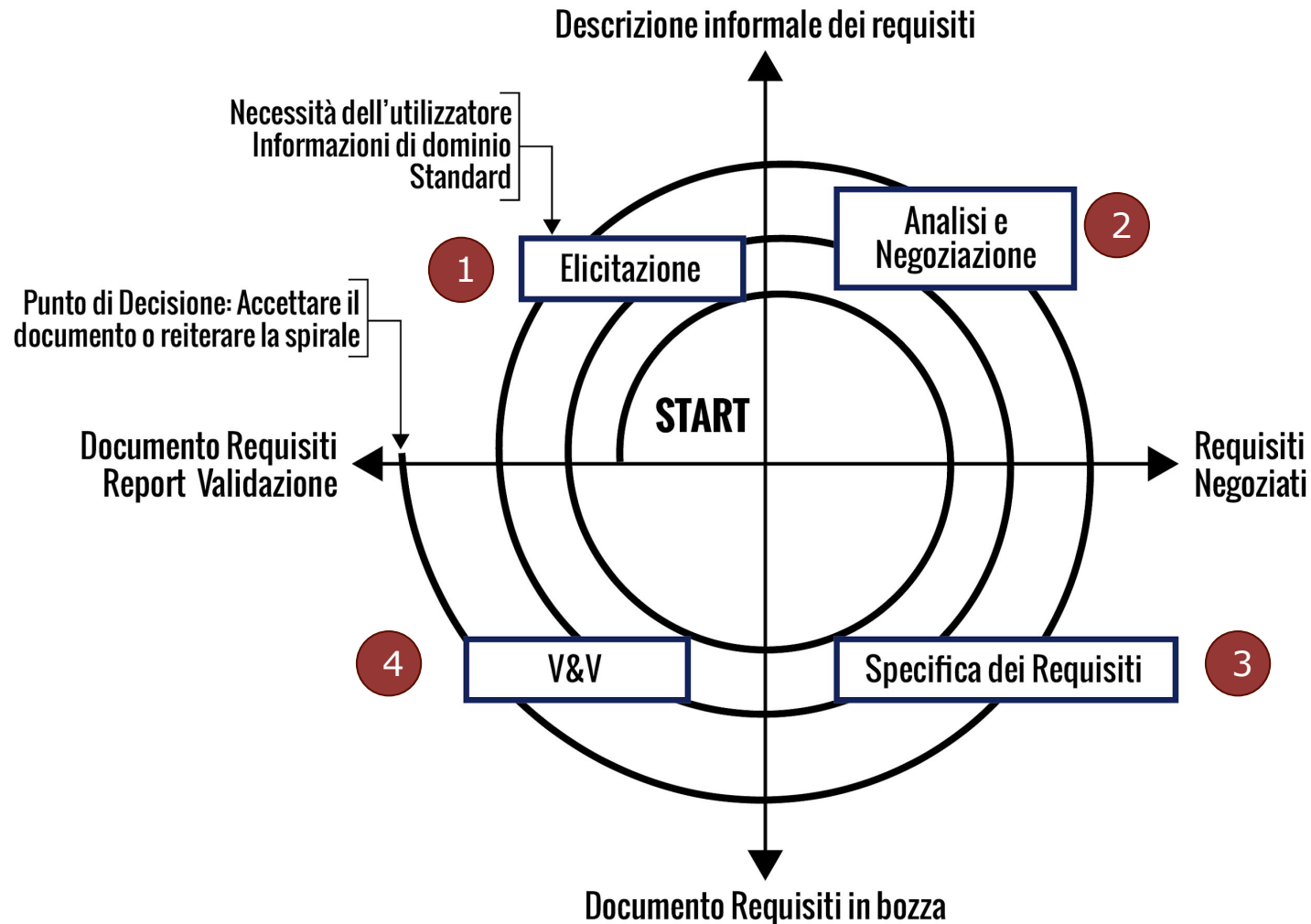
WORKING PRODUCT ONLY AT LATE STAGE

RECOMMEND FOR SHORT PROJECTS WITH EXACT SPECIFICATION

NOT RECOMMEND FOR LONG ONGOING OR COMPLEX PROJECTS



Schema di un Processo di Riferimento



1. Elicitazione: Sorgenti

- **Obiettivi di business**
 - ❑ Interessi del committente o fattori di successo
- **Conoscenza del dominio applicativo**
 - ❑ L'ingegnere del software lo deve conoscere indipendentemente dagli utenti, eventualmente coinvolti nelle attività
- **Parti interessate**
 - ❑ Classificazione della criticità delle parti interessate e dei relativi valori desiderati
- **Regolamenti o Leggi o Principi**
 - ❑ Definizione del comportamento che il sistema deve implementare
- **Ambiente operativo**
 - ❑ Ambiente in cui il sistema deve operare che, normalmente, genera vincoli. Per esempio: prestazioni, tempi, infrastrutture da utilizzare
- **Ambiente organizzativo**
 - ❑ Struttura, cultura e politiche interne dell'organizzazione



1. Elicitazione: Tecniche

- **Interviste**: metodo molto usato; ha notevoli limiti ed ha bisogno di molta abilità dell'intervistatore
- **Scenari**: modelli concettuali di diverso tipo; data flow, modelli di stato; interazione con gli utenti; modelli delle classi, modelli dei dati, ... In genere sono disponibili su strumenti appositi; p.e. UML
- **Prototipazione**: strumento per disambiguare requisiti che potrebbero nascondere ambiguità o che siano particolarmente critici
- **Opinioni** di stakeholders di sistemi analoghi
- **Ambienti operativi** (timing, interoperabilità)
- **Ambienti organizzativi** (cultura, logistica)
- **Osservazione**: analisi dell'ingegnere del software eseguita direttamente nell'ambiente operativo
- **Storie utente**: utilizzata normalmente nei processi agili, sono del tipo. Come <ruolo> desidero <goal> per avere <beneficio>
- **Studi di mappatura**: studi del mercato e della letteratura nel dominio applicativo



2. Analisi e Negoziazione

- Raccolti i requisiti è necessario farne un'analisi critica per:
 - ❑ Rilevare **conflitti** ed **inconsistenze**
 - ❑ Identificare i **requisiti mancanti**
- L'analisi produce un **report** in cui sono presenti le seguenti informazioni:
 - ❑ Parametri di Prodotto
 - **Funzioni** o **Capacità** o **Servizi** del Prodotto:
 - Cosa deve fare
 - Come si deve comportare
 - **Caratteristiche Tecniche**:
 - Qualità (prestazioni, manutenibilità, sicurezza, affidabilità...)
 - Vincoli (standard, legislativi, organizzativi, operativi...)
 - ❑ Parametri di Processo
 - Le condizioni da rispettare nello sviluppo (ad esempio: standard da utilizzare).
 - ❑ **Portata** di ogni requisito in termini di componenti su cui esso impatta. I requisiti che non sono allocabili sono proprietà emergenti ed influenzano la progettazione
 - ❑ **Priorità** dei requisiti
 - ❑ **Varianti** ed **invarianti**
 - ❑ **Volatilità** o stabilità
- **Negoziazione**. Risoluzione di eventuali conflitti tra i concetti espressi. Mitiga o azzera le dissonanze concettuali



3. Specifica dei requisiti

- **Specifica dei Requisiti** è la trasformazione dei requisiti in un **Modello Concettuale** che sia
 - ☐ Verificabile
 - ☐ Validabile
 - ☐ Facilmente trasformabile in architettura
- Per consentire la validazione dei requisiti il **Modello Concettuale** deve utilizzare termini e concetti caratteristici del Dominio Applicativo
- Per esprimere il Modello Concettuale si usano
 - ☐ **Linguaggi formali, simbolici** o **rappresentazioni grafiche**



Prodotto della Specifica dei requisiti

- Il prodotto della specificazione consiste nelle **Specifiche dei Requisiti Software (SRS)** che devono essere almeno **rigorose** e se possibile formali
- Per la produzione delle SRS si **usano strumenti** che possono dipendere da
 - ☐ I concetti da esprimere
 - ☐ L'esperienza dell'analista
 - ☐ I vincoli del committente
 - ☐ I tools disponibili

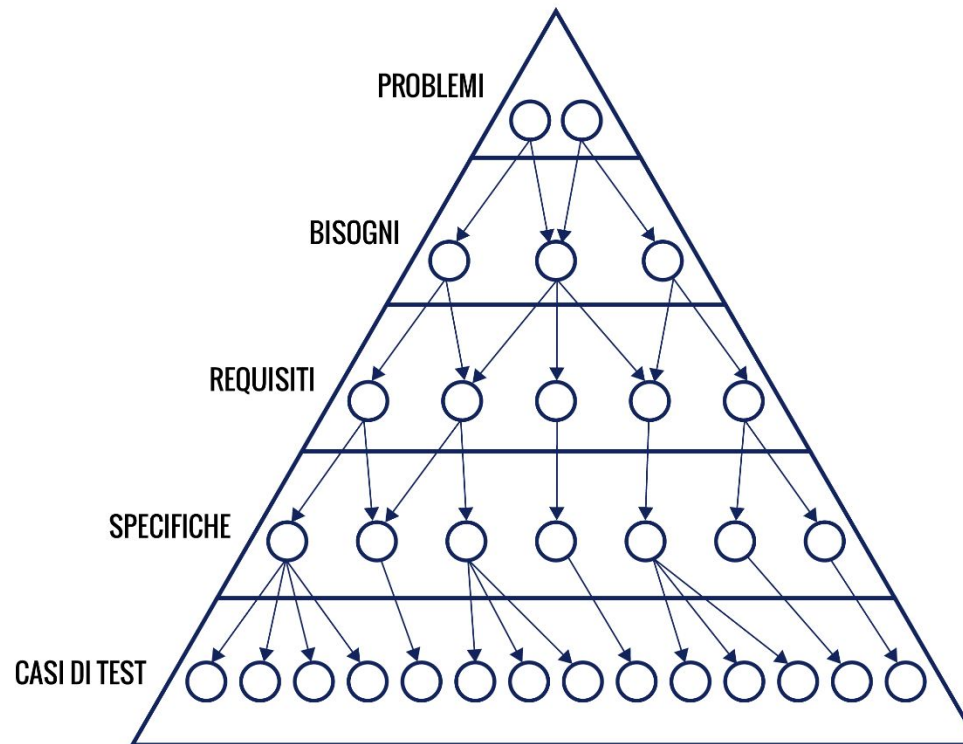
4. Verifica & Validazione (V&V)

- **Verifica dei Requisiti**: Conformità e correttezza del modello
 - ❑ **Ispezione**, per verificare il modello
- **Validazione dei Requisiti**: Corrispondenza dei contenuti alle richieste che il Sistema deve soddisfare
 - ❑ **Statica**
 - **Ispezione**, per validare il modello
 - ❑ **Dinamica**
 - Prototipi per validare i comportamenti più critici
 - Test di Accettazione per validare che il sistema finale soddisfi i requisiti richiesti



IL PROCESSO di ELICITAZIONE dei REQUISITI

La Piramide dell'Analisi



Individuazione dei Bisogni

- **Per ogni Problema è necessario trovare la causa**
 - ❑ Se non si trova la causa, **il problema non ha soluzione**
 - ❑ Se la causa non è eliminabile o moderabile il **problema non ha soluzione**
- Gli **accorgimenti** che si ritengono possano **eliminare o moderare la causa** costituiscono i **Bisogni**
- Questi ultimi spesso sono chiamati **Requisiti di Business**
 - ❑ **Il Bisogno descrive «cosa» risolve il problema**

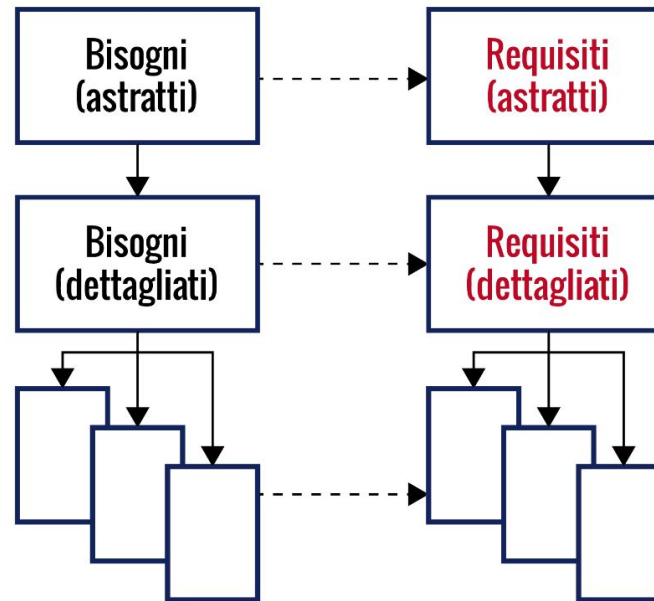
ELICITAZIONE: CONSIDERAZIONI

Bisogni vs Requisiti

- **Visione errata**: i Requisiti sono il dettaglio dei Bisogni
- **La differenza** tra Bisogni e Requisiti non è nel dettaglio ma **nei contenuti**:
 - ❑ I **Bisogni** esprimono il «**cosa**» serve per la soluzione del problema
 - ❑ I **Requisiti** esprimono il **Modello Concettuale** del Funzionamento del Sistema
- I **Bisogni** cambiano con il **Dominio dei Problemi**
- I **Requisiti** cambiano con il **Dominio delle Soluzioni**
 - ❑ I **Bisogni cambiano più lentamente** dei Requisiti
 - ❑ Operare con la chiara distinzione tra Bisogni e Requisiti consente di **gestire più efficientemente ed efficacemente lo Scorrimento dei Requisiti**, durante la produzione del sistema



Bisogni vs Requisiti



- La **mappatura tra Bisogni e Requisiti** è descritta dallo schema che rappresenta la relazione esistente
- La **mappatura** è tanto più **accurata** quanto migliore è il **bilanciamento** del dettaglio dei Bisogni e dei requisiti
- **Migliore mappatura** o tracciabilità induce un **più rapido soddisfacimento** dell'Utente

Bisogni vs Requisiti

- **I Bisogni non hanno mai cambiamenti piccoli:**
 - ❑ «cambiare il campo X» oppure
 - ❑ «aggiungere un attributo alla entità Ex»
- I Bisogni non sono presi in considerazione quando si acquista un Sistema Software
 - ❑ **Il potenziale cliente** di un Sistema Software spesso ha **difficoltà** nel verificare che i suoi Bisogni siano quelli che il venditore sostiene siano soddisfatti dal prodotto proposto
- Dimostrare che il sistema software proposto risolve o mitiga determinati tipi di problemi o realizza specifici obiettivi facilita la valutazione dell'efficacia del sistema nel conferire efficaci valori al business del compratore; quindi, l'acquisizione risulterebbe più facile



I Ruoli adeguati nella Elicitazione dei Requisiti

- **Tecniche di elicitazione** devono essere eseguite adeguatamente per **far partecipare** efficacemente l'utente o l'esperto del Dominio dei Problemi
- Il *Chirurgo*, l'*Ingegnere Navale*, il *Venditore* o qualsiasi altro **utente parla solo dei Problemi e dei Bisogni** nel linguaggio del Dominio dei Problemi di cui è esperto
 - possibile dissonanza concettuale
- Se si costringe l'utente a parlare o capire «tecnichese» non lo comprende, si annoia, **non partecipa** e si corrobora l'idea che *l'utente non sa quello che vuole*
 - ❑ Questo **non assicura una corretta individuazione dei Requisiti** nonostante l'utente sia un esperto del Dominio dei Problemi



I Ruoli adeguati nella Elicitazione dei Requisiti

- La **traduzione** in *tecnichese* dei Problemi e dei Bisogni è compito dell'**Analista**
- Un'inadeguata traduzione dei Bisogni nei Requisiti inietta difetti nei Concetti di Funzionamento del Sistema
 - ❑ Questi si trasformeranno in **danni sempre più rilevanti** durante le successive fasi della produzione realizzando **sprechi** ed **insoddisfazioni** degli utenti

VERIFICA: La QUALITÀ dei REQUISITI

Non Ambiguo

Deve esserci un solo modo di interpretare ogni requisito

- Le ambiguità sono create da:

- Gli **acronimi**:

- Req1: Richiedere al cliente di digitare il **PIN**. Cosa è il PIN? È opportuno scrivere il nome per intero e tra parentesi l'acronimo: Personal Identification Number (PIN)

- **Cattivo uso dei termini**:

- Req2: Il sistema dispensa contanti **fino a 500 € a scelta del cliente**; non è chiaro se:
 - La somma che deve dispensare è un numero digitato dal cliente tra 1 e 500€, con o senza decimali;
 - La somma è digitata dall'utente ma deve essere un multiplo di una certa cifra (ad esempio 50€);
 - La somma è scelta tra una lista proposta dalla macchina;
 - La somma digitata è rifiutata se si digitano dei decimali o viene dispensata la somma corrispondente alla parte intera;
 - La somma digitata è rifiutata nel caso che sia maggiore di 500€ o viene erogata comunque la somma di 500€.

Potrebbe essere corretto come:

- Req2: Il sistema dispensa una somma a scelta del cliente tra quelle proposte [100,150,200,250,300,400,500]

- **Eccessiva sintesi**:

- Req3: Il sistema mostra 5 movimenti di Deposito o di C/C quando il cliente chiede l'estratto conto
 - Significa che ne mostra solo 5 e quali: i primi, gli ultimi, 5 presi a caso?

Sarebbe corretto dire:

- **Req3**: il cliente richiede l'estratto conto con il numero (massimo 20) degli ultimi movimenti effettuati sul conto di deposito o di conto corrente che ha indicato; il sistema ne mostra 5 per videata; se il cliente ne chiede più che 20 il sistema modificherà il numero a 20 e avvertirà l'utente del taglio.

Provabile o verificabile

Deve essere possibile **costruire casi di test** che debbano essere corretti e non corretti rispetto ad ogni requisito per **verificare** che il sistema elabori correttamente i primi e rigetti i secondi

- Alcune parole o espressioni potrebbero rendere il requisito non provabile:
 - ❑ **Qualche Aggettivo**: robusto, sicuro, accurato, effettivo, efficiente, espandibile, flessibile, manutenibile, disponibile, amichevole, adeguato, ...
 - ❑ **Qualche avverbio**: velocemente, tranquillamente, tempestivamente, ...
 - ❑ **Parole ed acronimi non specifici**: ecc.; e/o; TBD, ...
 - ❑ **Parole generiche**: gestire, manipolare, ...
 - ❑ **Espressioni generiche**: che sia appropriato; come richiesto, se necessario, ...
 - ❑ **Pronomi indefiniti**: pochi, molti, tanto, spesso, qualche volta, tutti, qualsiasi, alcune, qualcuno, ...
 - ❑ **Voci passive**: il soggetto della espressione riceve l'azione del verbo invece che farla
- Esempio:
 - ❑ Req4: Il numero di conto digitato dal cliente sarà controllato **per esattezza ed esistenza** nella base dati
 - ❑ **Req4**: Il sistema controlla che il numero di conto digitato sia corretto sintatticamente ed esista nella base di dati



Chiaro (Conciso, laconico, semplice, preciso)

Il requisito non deve contenere verbosità o informazioni non necessarie

- Esempio:
 - ❑ Req5: Qualche volta il cliente potrebbe chiedere l'estratto conto del deposito o del conto corrente intestato a lui, in tal caso deve dichiarare il periodo a cui si deve riferire l'estratto conto richiesto ed è necessario chiedere se vuole la stampa su carta o gli basta vederlo sul video
 - ❑ **Req5:** Il cliente può chiedere l'estratto conto per il periodo che dichiara; il sistema riporta l'estratto conto, a scelta del cliente, su carta o su video



Corretto

Se un requisito contiene fatti, questi devono essere **veri**

- Esempio:

- ❑ ReqE: Il **costo** dell'operazione sarà **di 1 €**
- ❑ **Non è corretto** per almeno due motivi:
 - Le operazioni eseguibili all'ATM non possono avere tutte lo stesso prezzo
 - Il prezzo delle operazioni dipendono dalla politica della Banca che gestisce l'ATM e dagli accordi di questa con la Banca che ha emesso la carta utilizzata dal cliente

Comprensibile

I requisiti devono essere grammaticalmente corretti e scritti in stile consistente

- Devono essere usati appositi standard
- La parola “**deve**” sarà utilizzata invece di “**volere**”, “**bisogna**” o “**può**”

Fattibile (Realistico, Possibile)

I requisiti devono essere **realizzabili** con i vincoli esistenti come tempo, denaro e risorse disponibili

- Esempio:
 - ReqF: Il sistema userà un linguaggio naturale nell'interfaccia così che comprenda i comandi espressi in lingua italiana
 - Questo requisito richiede **molto tempo** e un **grande investimento** con **notevole rischio** di non essere realizzato con un buon livello di affidabilità

Indipendente-Auto-consistente

Per comprendere un requisito non deve essere necessario conoscere nessun altro requisito

- Esempio:
 - ❑ ReqI: Il sistema elenca tutti gli importi erogabili per il cliente che ha inserito la carta di credito nell'ATM
 - ❑ ReqD: **Essi** sono elencati in ordine crescente
 - "Essi" si riferisce agli importi erogabili di R_i , se l'ordine dei requisiti dovesse modificarsi nello SRS allora questo requisito non sarebbe comprensibile
 - Perciò questo è un requisito da rivedere



Atomico

Ogni requisito deve contenere un solo elemento tracciabile

- Le espressioni che contengono “**e**” o “**ma**” devono essere riviste e controllate
- Esempio:
 - ❑ ReqA: Il cliente **inserisce il PIN**, chiede **l’erogazione di una somma e l’estratto conto**
 - ❑ Questo requisito ne contiene tre atomici

Necessario

- Un requisito è **inutile** se:
 - ❑ Nessuna parte interessata ha bisogno di esso
 - ❑ La **cancellazione** del requisito non ha **alcuna conseguenza** sul sistema finale perché non aggiunge alcuna informazione
- Requisiti inutili sono quelli che l'**analista inserisce** nello SRS perché ritiene che siano desiderati dalle parti interessate ma **nessuno** di questi li ha chiesti
- Esempi:
 - ❑ ReqI: Tutti i requisiti specificati nello SRS devono essere testati
 - ❑ ReqL: Il sistema stampa il nome della filiale che gestisce l'ATM utilizzato dal cliente

Astratto – indipendente dalla implementazione

- I requisiti astratti non contengono dettagli circa la loro implementazione
 - ❑ Salvo che non sia un vincolo che l'utente vuole dichiarare
- L'implementazione è di interesse del **progettista** e non interessa agli utenti del sistema
- Esempio:
 - ❑ ReqA: Il contenuto informativo sarà memorizzato in forma strutturata



Consistente (1/3)

Tutti i requisiti devono utilizzare termini uguali per esprimere concetti uguali e nessun requisito deve essere conflittuale con altri requisiti

- I **conflitti** possono essere
 - ❑ **Diretti**: Quando in una stessa situazione il comportamento del sistema deve essere diverso
 - ❑ **Indiretti**: Quando due requisiti non descrivono la stessa situazione ma non è possibile soddisfarli contemporaneamente
- Esempio di **termini inconsistenti**
 - ❑ ReqX: L'**ATM** erogherà la cifra richiesta dal **cliente**
 - ❑ ReqY: Il **Bancomat** stampa l'estratto conto richiesto dall'**utente**
 - Per indicare il sistema i due requisiti utilizzano **due termini diversi**
 - Per indicare l'utilizzatore del sistema utilizzano due termini diversi



Consistente (2/3)

- Esempio di **conflitto diretto**:

- ❑ ReqX: L'ATM accetta tutte le carte di credito ed il Bancomat emesso da una qualsiasi banca
- ❑ ReqY: L'ATM accetta i Bancomat emessi dalle banche convenzionate con la Banca gestore
 - Per eliminare il conflitto è necessario **cancellare uno dei due requisiti**
- ❑ ReqX: Le date devono essere visualizzate nella forma mm/dd/yyyy
- ❑ ReqY: Le date devono essere visualizzate nella forma gg/mm/aaaa
 - Questi requisiti oltre ad utilizzare termini inconsistenti hanno un conflitto diretto perché richiedono formati diverso per uno stesso tipo di dato

Si possono **correggere** facendo una **precisazione**:

- ❑ ReqX: Le date per gli ATM in U.S. devono essere visualizzate nella forma mm/dd/yyyy
 - ❑ ReqY: Le date per gli ATM in Italia devono essere visualizzate nella forma gg/mm/aaaa
- Potrebbe essere eliminato il conflitto **generalizzando** il requisito:
- ❑ ReqG: Le date saranno visualizzate nel formato definito dall'utente all'installazione

Consistente (3/3)

- Esempio di **conflitto indiretto**:
 - ❑ ReqX: Il sistema deve avere l'interfaccia in linguaggio naturale
 - ❑ ReqY: Il sistema sarà prodotto in tre mesi
 - Chiaramente non è possibile soddisfarli entrambi
 - Per eliminare il conflitto è necessario rilassare uno dei due

Ridondanza

Ogni requisito deve essere espresso solo una volta e non deve sovrapporsi ad un altro

- Esempio:
 - ❑ ReqX: Il sistema mette a disposizione un calendario per aiutare a definire le date di intervallo dell'estratto conto richiesto
 - ❑ ReqY: Il sistema mette a disposizione un calendario ogni volta che si deve digitare una data
 - ❑ **ReqX è un caso particolare di ReqY**, perciò il primo deve essere cancellato



Completezza

Un requisito deve essere specificato per tutte le condizioni che possono verificarsi

- Esempio:

- ❑ ReqX: Il sistema non visualizza la data di prelievo
- ❑ ReqY: Il sistema non visualizza l'anagrafica del cliente

Il numero del conto che non è né la data né l'anagrafica, **lo visualizza?**

Caratteristiche derivate

- **Manutenibile**
 - ❑ Un requisito atomico e non ridondante è più facilmente modificabile
- **Tracciabile**
 - ❑ Un requisito atomico e con un identificatore univoco è tracciabile

Riepilogando: LA QUALITÀ DEI REQUISITI

- Non Ambiguo
- Provabile o verificabile
- Chiaro (Conciso, laconico, semplice, preciso)
- Corretto
- Comprensibile
- Fattibile (Realistico, Possibile)
- Indipendente-Auto-consistente
- Atomico
- Necessario
- Astratto – indipendente dalla implementazione
- Consistente
- Non Ridondante
- Completezza
- Caratteristiche derivate: **Manutenibile** e **Tracciabile**

Riferimenti

- Ian Sommerville “Ingegneria del Software”, 10a ed. Pearson, 2017
 - Capitolo 4: Ingegneria dei requisiti
 - 4.1 Requisiti funzionali e non funzionali
 - 4.2 Processi di ingegneria dei requisiti
 - 4.3 Deduzione dei requisiti
 - 4.4 Specifica dei requisiti
 - 4.5 Convalida dei requisiti

